

**Universidade da Beira
Interior**
Departamento de Informática



**Departamento de
Informática**

Grupo N^o 4 : *[NestPet]*

Elaborado por:

**Rodrigo Almeida : n^o 51597
Hugo Serra : n^o 52297
João Freire : n^o 52216
Alexandre Machado : n^o 51882**

Orientador:

Professor Doutor Paulo André Pais Fazendeiro

8 de dezembro de 2025

Resumo

O projeto *NestPet* é uma plataforma digital que facilita a gestão e visualização de animais para adoção. A aplicação oferece dois perfis distintos: **Utilizador** (pessoas que procuram adotar animais) e **Organização** (associações ou clínicas que gerenciam animais disponíveis). Os utilizadores podem navegar por um catálogo filtrado de animais, visualizar detalhes com fotos e vídeos, enquanto as organizações têm acesso a ferramentas de CRUD para gerenciar seus animais, incluindo tags visuais de cor e personalidade. Desenvolvida em Flutter com backend em Supabase, a aplicação assegura autenticação segura via JWT, interface responsiva com temas personalizados (#824822 e #F2E8D7) e comunicação em tempo real através de WebSocket.

Capítulo 1

Introdução

1.1 Descrição da Proposta

O NestPet é uma aplicação móvel que tem como objetivo facilitar o encontro entre animais disponíveis para adoção e potenciais tutores. A plataforma distingue dois tipos de utilizadores: pessoas individuais (Utilizador) e entidades (Organização). Os utilizadores podem explorar um catálogo de animais, aplicar filtros por espécie, idade, peso e características visuais (cor/personalidade) e visualizar detalhes com fotos e vídeos. As organizações têm acesso a uma área dedicada onde podem adicionar, editar e remover animais, atribuindo-lhes tags visuais que facilitam a identificação. A aplicação foi desenvolvida com Flutter (frontend) e Supabase (backend), garantindo autenticação segura, uma interface coerente entre Android e iOS (em teoria funciona em iOS, mas devido a falta de recursos, não foi possível testar essa funcionalidade) e a possibilidade de expansão futura para funcionalidades como chat ou sistema de adoções.

1.2 Organização do Documento

Este relatório está dividido em cinco capítulos principais:

- O primeiro capítulo (**Introdução**) apresenta o problema, o contexto e os objetivos.
- O segundo capítulo (**Engenharia de Software**) descreve requisitos, diagramas e planeamento.
- O terceiro capítulo (**Implementação**) aborda decisões técnicas, arquitetura e detalhes de construção da aplicação.
- O quarto capítulo (**Reflexão Crítica**) analisa resultados, dificuldades e melhorias futuras.
- O quinto capítulo (**Conclusão**) resume o que de melhor tiramos deste trabalho.

Capítulo 2

Engenharia de Software

2.1 Ferramentas e Tecnologias Utilizadas

- **Frontend/Mobile:**
 - Flutter (v3.10 ou superior) - Framework multiplataforma
 - Dart - Linguagem de programação
 - Material Design 3 com tema personalizado (#824822, #F2E8D7)
- **Backend & Base de Dados:**
 - Supabase - Plataforma Backend-as-a-Service
 - PostgreSQL - Sistema de base de dados relacional
 - Autenticação JWT com RLS (Row Level Security)
 - APIs RESTful e WebSocket para atualizações em tempo real
- **Gestão de Estado & Navegação:**
 - Provider - Gestão de estado global
 - GoRouter - Navegação declarativa entre ecrãs
- **Multimédia:**
 - video_player - Reprodução de vídeos nos detalhes dos animais
 - image_picker - Seleção de fotos para upload
- **Ambiente de Desenvolvimento:**
 - VS Code / Android Studio - IDEs principais
 - Flutter SDK - Ferramentas CLI para build e emulação
 - Git - Controlo de versões
 - GitHub - Hospedagem do repositório

2.2 Requisitos

Requisitos Funcionais (Implementados)

- **RF1:** Autenticação segura com dois papéis (Utilizador/Organização)
- **RF2:** Navegação por catálogo de animais com filtros (espécie, idade, peso, cor, personalidade)
- **RF3:** Visualização detalhada de animais com fotos e vídeos
- **RF4:** CRUD completo de animais para organizações
- **RF5:** Atribuição de tags visuais de cor e personalidade aos animais
- **RF6:** Gestão de perfil de utilizador e organização
- **RF7:** Interface adaptada ao tipo de perfil (fluxos separados)

Requisitos Não Funcionais

- **RNF1:** Interface simples, intuitiva e acessível com tema coerente
- **RNF2:** Tempo de resposta inferior a 2 segundos para operações principais
- **RNF3:** Segurança dos dados utilizando autenticação JWT via Supabase Auth
- **RNF4:** Aplicação responsiva e compatível com Android e iOS a partir de uma base de código
- **RNF5:** Comunicação em tempo real via WebSocket para atualizações do catálogo
- **RNF6:** Escalabilidade através da infraestrutura gerida do Supabase

2.3 Casos de Uso (Implementados)

UC1 — Autenticação no Sistema

Descrição: O utilizador faz login com email e password. As organizações têm um fluxo de registo que inclui NIF e nome da entidade.

Atores: Utilizador, Organização.

Pré-condições: Conta criada no sistema.

Pós-condições: Acesso concedido conforme papel, com interface adaptada.

UC2 — Navegar no Catálogo

Descrição: O utilizador visualiza uma lista de animais disponíveis, aplica filtros (espécie, idade, peso, cor, personalidade) e acede à página de detalhe.

Atores: Utilizador.

Pré-condições: Autenticação bem-sucedida como Utilizador.

Pós-condições: Lista filtrada apresentada; possível visualização de detalhes.

UC3 — Visualizar Detalhe do Animal

Descrição: O utilizador vê informações detalhadas sobre um animal, incluindo fotos, vídeos, tags de cor/personalidade e dados básicos.

Atores: Utilizador.

Pré-condições: Animal selecionado a partir do catálogo.

Pós-condições: Informação completa apresentada.

UC4 — Gerir Animais (Organização)

Descrição: A organização adiciona, edita ou remove animais do sistema, atribuindo tags de cor e personalidade.

Atores: Organização.

Pré-condições: Autenticação bem-sucedida como Organização.

Pós-condições: Animal adicionado/editado/removido do catálogo.

UC5 — Editar Perfil

Descrição: O utilizador ou organização atualiza informações pessoais/entidade (nome, email, foto, etc.).

Atores: Utilizador, Organização.

Pré-condições: Utilizador autenticado.

Pós-condições: Dados atualizados localmente e no servidor.

2.4 Conclusão da Engenharia de Software

A fase de engenharia de software do projeto **NestPet** permitiu estabelecer as bases sólidas para uma aplicação móvel funcional e escalável. A análise de requisitos resultou numa especificação clara das funcionalidades implementadas: autenticação dual, catálogo com filtros avançados, visualização detalhada com multimédia, e gestão completa de animais para organizações.

A escolha tecnológica (**Flutter** + **Supabase** + **Provider**) mostrou-se adequada, oferecendo:

- **Produtividade** com desenvolvimento multiplataforma a partir de código único
- **Segurança e escalabilidade** através da infraestrutura gerida do Supabase (JWT, RLS)
- **Manutenibilidade** com arquitetura baseada em providers e separação de responsabilidades

Os casos de uso refletem os fluxos reais da aplicação, garantindo que as necessidades dos dois perfis de utilizador (individual e organização) foram corretamente mapeadas. A diferenciação de interfaces e permissões conforme o papel foi um aspeto central da especificação.

Os requisitos não funcionais estabelecem padrões de qualidade que orientaram o desenvolvimento: performance, segurança, responsividade e experiência de utilizador coerente através do tema visual definido (#824822, #F2E8D7).

Em suma, esta fase forneceu um guia claro e realista para a implementação, alinhando expectativas com as capacidades reais da stack tecnológica escolhida e garantindo que o produto final corresponde aos objetivos definidos.

Capítulo 3

Implementação

3.1 Escolhas de Implementação

A escolha do Flutter e Dart em detrimento do Java para o desenvolvimento do Nest-Pet foi fundamentada em critérios de produtividade, consistência multiplataforma e integração tecnológica. O Flutter permitiu desenvolver uma única base de código para Android e iOS, reduzindo duplicação de esforços e acelerando o ciclo de desenvolvimento através do *hot reload*. A linguagem Dart facilitou a gestão de operações assíncronas necessárias para comunicação com o Supabase, enquanto o sistema de widgets declarativos simplificou a criação de interfaces reativas.

Tecnicamente, o Flutter ofereceu performance nativa, um ecossistema maduro com pacotes essenciais (como `supabase_flutter` e `provider`) e consistência visual entre plataformas — vantagens decisivas face ao desenvolvimento Android nativo com Java. A curta curva de aprendizagem para a equipa e a robusta documentação disponível consolidaram esta escolha, assegurando a entrega do projeto dentro dos prazos estabelecidos sem comprometer a qualidade da aplicação final.

3.2 Arquitetura e Layout

Arquitetura de Duas Apps em Uma

A aplicação foi concebida como uma única base de código que contém duas “sub-aplicações” distintas conforme o papel do utilizador:

- **Fluxo do Utilizador:** Focado em navegação, filtros e visualização
- **Fluxo da Organização:** Focado em gestão (CRUD) de animais

A distinção é feita através de rotas protegidas (`GoRouter`) e controlo de estado (`Provider`), garantindo que cada perfil acede apenas às funcionalidades permitidas.

Tema Visual Coerente

Foi implementado um tema personalizado com as cores:

- **Cor principal:** #824822 (tom terroso)

- **Cor de fundo:** #F2E8D7 (bege claro)

Este esquema aplica-se de forma consistente em todos os ecrãs, incluindo botões, barras de navegação e indicadores de seleção.

3.3 Detalhes de Implementação

3.3.1 Arquitetura do Sistema

O sistema segue uma arquitetura em três camadas:

Frontend (Flutter):

- Estrutura baseada em widgets reativos
- Gestão de estado com Provider
- Navegação via GoRouter com rotas protegidas por papel
- Temas personalizados (cores #824822 e #F2E8D7)
- Componentes reutilizáveis para filtros, tags e cards de animais

Backend (Supabase):

- Autenticação JWT com roles (user/org)
- Base de dados PostgreSQL com RLS (Row Level Security) ativado
- API REST automática para operações CRUD
- Storage Bucket para upload de fotos e vídeos dos animais
- WebSocket para atualizações em tempo real do catálogo

Comunicação:

- SDK oficial `supabase_flutter`
- Sincronização automática de dados
- Upload de multimédia diretamente para Supabase Storage

3.3.2 Estrutura de Código

```
lib/
  main.dart                # Ponto de entrada da aplicação
  app_router.dart          # Configuração de rotas (GoRouter)
  supabase_config.dart     # Configuração do cliente Supabase
  assets/
    NestPet_Icon.png
    NestPet_logo.png

  models/                  # Modelos de dados
    animal.dart            # Modelo Animal (espécie, raça, idade, etc.)
    message.dart           # Modelo Message (chat)

  providers/               # Gestão de estado (Provider)
    app_state.dart         # Estado global (auth, animais, favoritos)

  data/                    # Repositórios de dados
```



```

animal_repository.dart          # Interface base de animais
storage_repository.dart         # Upload de media
supabase_animal_repository.dart # CRUD de animais
supabase_chat_repository.dart   # Chat em tempo real
supabase_favorites_repository.dart # Sistema de favoritos

services/                        # Serviços auxiliares
  session_service.dart          # Persistência de sessão local
  auth_service.dart             # Autenticação e gestão de utilizadores

screens/                         # Ecrãs da aplicação
  auth/                         # Autenticação
    welcome_screen.dart         # Ecrã inicial
    login_screen.dart           # Login
    register_user_screen.dart   # Registo utilizador
    register_org_screen.dart    # Registo organização

  user/                         # Ecrãs de utilizador
    user_home_screen.dart       # Home com grid de animais
    animal_detail_screen.dart   # Detalhes do animal
    animal_filters_sheet.dart   # Bottom sheet de filtros
    user_favorites_screen.dart  # Lista de favoritos
    profile_screen.dart         # Perfil do utilizador
    chat_screen.dart            # Chat com organização

  org/                          # Ecrãs de organização
    my_animals_screen.dart      # Lista/gestão de animais da org
    add_animal_screen.dart      # Adicionar animal
    edit_animal_screen.dart     # Editar animal
    edit_org_screen.dart        # Editar dados da organização
    profile_screen.dart         # Perfil da organização
    chat_screen.dart            # Chat com utilizadores

  common/                       # Ecrãs partilhados
    edit_account_screen.dart    # Editar nome/avatar
    messages_screen.dart        # Lista unificada de mensagens

  widgets/                      # Widgets reutilizáveis
    animal_grid_card.dart       # Card de animal no grid
    empty_state.dart            # Estado vazio genérico

  widgets/                      # Widgets globais
    avatar_picker.dart          # Seletor de avatar com câmara/galeria
    change_password_flow.dart   # Fluxo de alteração de password

utils/                          # Utilitários
  color_tags.dart              # Lista de cores disponíveis
  personality_tags.dart        # Lista de personalidades
  unsaved_changes_guard.dart    # Sistema de avisos de alterações não guardadas

```

```
assets/                                # Recursos estáticos
(imagens, fontes, etc.)
```

3.3.3 Fluxos Principais Implementados

1. Autenticação e Separação de Papéis:

- Registo com validação de email e seleção de papel (user/org)
- Login persistente com recuperação de sessão
- Logout com limpeza de tokens
- Redirecionamento automático para o fluxo correto conforme papel

2. Catálogo e Filtros:

- Lista paginada de animais com scroll infinito
- Filtros por espécie, idade, peso, cor e personalidade
- Sistema de tags visuais para cores e características
- Botão de "limpar filtros" para reset rápido

3. Gestão de Animais (Organização):

- Formulário completo para adicionar/editar animais
- Upload de múltiplas fotos e um vídeo por animal
- Seleção de tags de cor e personalidade
- Validação em tempo real dos campos obrigatórios

3.3.4 Segurança Implementada

- Tokens JWT com expiração geridos automaticamente pelo Supabase
- RLS (Row Level Security) no PostgreSQL para garantir isolamento de dados
- Validação de inputs no frontend (campos obrigatórios, formatos)
- Proteção de rotas conforme papel do utilizador
- Sanitização de queries através do cliente Supabase

3.4 Manual de Instalação

Pré-requisitos

- Flutter SDK versão 3.10 ou superior
- Android Studio ou Visual Studio Code (com extensão Flutter)
- Conta Supabase com projeto criado
- Git instalado
- Dispositivo Android/iOS físico ou emulador configurado

Passos de Instalação

1. Clonar o repositório:

```
1 git clone [https://github.com/JadfPT/NestPet]
```

2. Instalar dependências:

```
1 flutter pub get
```

3. Executar a aplicação:

```
1 flutter run
```

3.5 Manual de Utilização

1. Primeiro Acesso

- Abrir a aplicação NestPet
- Selecionar "Registar" para criar nova conta
- Escolher tipo de conta:
 - **Utilizador:** Para pessoas que procuram animais para adoção
 - **Organização:** Para associações ou clínicas (requer NIF e nome da entidade)
- Preencher dados de registo e confirmar email
- Fazer login com as credenciais criadas

2. Navegar no Catálogo (Utilizador)

- Após login, verá a lista de animais disponíveis
- Utilizar a barra de filtros para:
 - Selecionar espécie (cão, gato, etc.)
 - Definir intervalo de idade e peso
 - Escolher cores e características de personalidade
- Clicar em "Limpar Filtros" para voltar à vista completa
- Selecionar um animal para ver detalhes completos

3. Visualizar Detalhes do Animal

- Na página de detalhe, ver:
 - Galeria de fotos (deslizar horizontalmente)
 - Vídeo do animal (se disponível)
 - Informações básicas (nome, idade, peso)
 - Tags de cor e personalidade (ícones visuais)
 - Descrição completa
- Utilizar botões de navegação para voltar à lista

4. Gerir Animais (Organização)

- Após login como organização, aceder à área "Meus Animais"
- Clicar no botão "+" para adicionar novo animal
- Preencher formulário:
 - Dados básicos (nome, espécie, idade, peso)
 - Selecionar fotos (até 10) e opcionalmente um vídeo
 - Escolher tags de cor e personalidade
 - Escrever descrição detalhada
- Guardar para publicar no catálogo
- Para editar ou remover: seleccionar animal da lista e usar opções

5. Gerir Perfil

- Clicar no ícone de perfil (canto superior direito)
- Editar informações pessoais ou da organização
- Alterar foto de perfil
- Atualizar dados de contacto
- Terminar sessão quando necessário

Funcionalidades por Tipo de Conta

- **Utilizador:** Apenas visualização, filtros e navegação
- **Organização:** Todas as funcionalidades anteriores + CRUD completo de animais

3.6 Conclusão da Implementação

O projeto NestPet foi implementado com sucesso utilizando uma stack tecnológica moderna e coerente. A arquitetura de "duas apps em uma" permitiu servir dois perfis de utilizador distintos a partir do mesmo código, garantindo eficiência no desenvolvimento e manutenção. As funcionalidades principais (autenticação dual, catálogo com filtros avançados, visualização detalhada com multimédia, gestão completa de animais e mensagens em tempo real) estão operacionais e alinhadas com os requisitos definidos. A integração com Supabase forneceu uma infraestrutura backend segura e escalável sem necessidade de desenvolvimento personalizado de APIs, acelerando significativamente o ciclo de desenvolvimento.

Capítulo 4

Reflexão Crítica e Problemas Encontrados

4.1 Objetivos Propostos vs. Alcançados

- **Totalmente Alcançados:**
 - Sistema de autenticação com dois papéis (utilizador/organização)
 - Catálogo de animais com filtros avançados (espécie, idade, peso, cor, personalidade)
 - Visualização detalhada com fotos e vídeo
 - Conversas em tempo real
 - CRUD completo de animais para organizações
 - Interface adaptada a cada perfil com tema coerente
 - Integração completa com Supabase (autenticação, base de dados, storage)
- **Adicionados Não Planeados:**
 - Sistema de tags visuais para cores e características
 - Upload simultâneo de múltiplas fotos e vídeo
 - WebSocket para atualizações em tempo real do catálogo
 - Botão de "limpar filtros" para melhor UX

4.2 Divisão do Trabalho

A distribuição das componentes do projeto foi organizada da seguinte forma:

- **Rodrigo Almeida:**
 - **UI/UX & Navegação** — Desenho dos ecrãs e percurso do utilizador na app
 - **Relatório Técnico** — Documentação completa do projeto

- **Hugo Serra:**
 - **Modelo de Dados & Repositórios** — Estrutura da informação e gestão de dados
 - **Integração Backend** — Configuração Supabase, RLS, queries
- **João Freire:**
 - **Catálogo, Filtros e Detalhe** — Lista de animais, pesquisa e páginas de detalhe
 - **Gestão de Animais (Instituição)** — CRUD de anúncios para organizações
- **Alexandre Machado:**
 - **Qualidade e Demo** — Testagem intensiva e revisão
 - **Chat** — Sistema de conversa entre utilizador e organização, em tempo real
 - **Autenticação** — Sistema de criação de conta, início/fim de sessão e gestão de perfis

Colaboração: Todas as componentes foram desenvolvidas com revisão cruzada. A integração frontend-backend e o sistema de filtros/tags foram trabalhados colaborativamente para garantir coerência.

4.3 Problemas Encontrados

- **Upload de Multimédia em Lote:** Dificuldade em gerir o upload simultâneo de múltiplas fotos e vídeo para o Supabase Storage, especialmente com controlo de progresso e tratamento de falhas.
- **Sincronização de Filtros com Estado Global:** Complexidade em manter os filtros aplicados consistentes entre diferentes ecrãs, exigindo uma gestão cuidadosa do estado partilhado.
- **Diferenciação de Interfaces por Papel:** Desafio em criar fluxos de UI significativamente diferentes (utilizador vs. organização) dentro da mesma aplicação, mantendo a coesão do código.

4.4 Reflexão Crítica

O projeto permitiu consolidar competências técnicas em desenvolvimento full-stack com tecnologias modernas (Flutter, Supabase, Provider) e melhorar substancialmente o trabalho em equipa através de metodologias colaborativas. Aprendemos a importância de:

- Definir APIs claras entre frontend e backend desde o início
- Estabelecer convenções de código consistentes para facilitar a colaboração
- Utilizar ferramentas de debugging específicas do Flutter (DevTools)
- Gerir dependências e versões de pacotes para evitar conflitos

No entanto, reconhecemos que a gestão temporal das tarefas poderia ter sido mais eficiente com um planeamento mais detalhado das dependências técnicas e uma distribuição mais equilibrada de cargas de trabalho ao longo do cronograma. A fase de integração consumiu mais tempo do que o previsto devido à necessidade de alinhar diferentes abordagens de implementação.

4.5 Conclusão da Reflexão

A equipa demonstrou evolução técnica significativa ao longo do projeto, superando desafios complexos de integração e arquitetura. Conseguimos entregar um produto funcional, coerente e com valor potencial para utilizadores reais. As competências adquiridas (tanto técnicas como de trabalho em equipa) representam um ganho duradouro para todos os elementos.

Capítulo 5

Conclusões e Trabalho Futuro

5.1 Conclusões Principais

O desenvolvimento do *NestPet* revelou-se uma experiência acadêmica e técnica enriquecedora, permitindo aplicar metodologias fundamentais de engenharia de software. Desde a análise de requisitos até à implementação com tecnologias modernas como Flutter e Supabase. A aplicação final demonstra que, apesar das limitações de tempo e recursos, é possível criar um protótipo funcional que resolve problemas reais de ligação entre animais disponíveis para adoção e potenciais tutores.

O projeto reforçou a importância de uma organização rigorosa, planeamento iterativo e comunicação clara entre membros da equipa. Competências que se revelaram tão cruciais quanto as técnicas para o sucesso da entrega. A escolha da stack tecnológica mostrou-se acertada, proporcionando um equilíbrio entre produtividade, performance e capacidade de manutenção.

5.2 Trabalho Futuro

Apesar do estado funcional atual, o *NestPet* possui margem significativa para evolução. As seguintes direções são sugeridas para desenvolvimento futuro:

- **Processo de Adoção Formal:** Criar um fluxo completo de adoção, com pedidos, aprovações, contratos e acompanhamento pós-adoção.
- **Lembretes e Histórico Clínico:** Adicionar funcionalidades de gestão de saúde animal: lembretes de vacinas, consultas, e registo de histórico médico.
- **Integração com APIs Externas:** Conectar a serviços de mapas para localização de clínicas veterinárias ou a sistemas de microchipagem nacional.
- **Análise de Dados e Relatórios:** Implementar dashboards administrativos com estatísticas de adoções e gerar relatórios PDF partilháveis.
- **Funcionalidades Sociais:** Criar uma comunidade onde tutores possam partilhar experiências, recomendar profissionais ou marcar encontros para socialização de animais.
- **Modo Offline Avançado:** Melhorar o suporte offline com sincronização bidirecional inteligente e cache de dados essenciais.

Estas expansões transformariam o NestPet de um catálogo básico numa plataforma completa de gestão do ciclo de vida da adoção animal, agregando ainda mais valor para todos os intervenientes no processo.

Bibliografia

- [1] Google, *Flutter Documentation*. Disponível em: <https://flutter.dev/docs>.
- [2] Supabase, *Supabase Documentation*. Disponível em: <https://supabase.com/docs>.
- [3] Remi Rousselet, *Provider Package*. Disponível em: <https://pub.dev/packages/provider>.
- [4] Flutter Community, *go_router Package*. Disponível em: https://pub.dev/packages/go_router.
- [5] Flutter Team, *video_player Package*. Disponível em: https://pub.dev/packages/video_player.
- [6] Supabase, *supabase_flutter Package*. Disponível em: https://pub.dev/packages/supabase_flutter.
- [7] Flutter Team, *image_picker Package*. Disponível em: https://pub.dev/packages/image_picker.
- [8] Google, *Material Design 3*. Disponível em: <https://m3.material.io>.